REACTJS FUNDAMENTALS — UNIT 3

Complete Exam-Ready Study Resource & Revision Manual
Course: Front-End Web Technology (FEWT - 2504CS305) | **Dept:** Computer Application Department
Faculty: Prof. Dharmikkumar G. Aghera (dharmikkumar.aghera@darshan.ac.in)

1. Introduction to ReactJS & Virtual DOM

1.1 Core Definition & Purpose

ReactJS is a **declarative, efficient, and flexible JavaScript library** developed by Facebook for building user interfaces, specifically tailored for **Single Page Applications (SPAs)**. Unlike traditional monolithic web frameworks, React operates strictly on the **View layer** (in MVC architecture) and enables developers to construct complex, interactive UIs out of small, isolated, and reusable code blocks called **components**.

1.2 Historical Milestones & Versioning

Milestone / Metric	Details / Historical Fact
Initial Facebook Usage	First deployed in 2011 for Facebook's **NewsFeed** feature.
Public Open-Source Release	Version 0.3.0 publicly released in **July 2013**.
Current Version Reference	**Version 18.2.0** (as referenced in course materials, Jan-2024).
Core Maintainer & Creator	Created and maintained by **Facebook (Meta)** alongside an open-source community.

1.3 Features, Advantages, Disadvantages & Applications

Key Features of ReactJS:

- **Declarative UI:** React makes it painless to create interactive UIs by designing simple views for each application state.
- **Component-Based Architecture:** Build encapsulated components that manage their own state, then compose them into complex UIs.
- **Virtual DOM:** Uses an in-memory light representation of the real DOM to optimize rendering speed and performance.
- **JSX Syntax:** An XML/HTML-like syntax extension to JavaScript that streamlines UI template writing.
- **Unidirectional Data Flow:** Data flows top-down from parent components to child components via read-only *props*.

Advantages vs Disadvantages:

Advantages	Disadvantages / Trade-offs
High rendering speed due to Virtual DOM diffing.	Covers only the View layer; requires extra libraries for routing/state.

Advantages	Disadvantages / Trade-offs
Reusable component hierarchy reduces code duplication.	Fast pace of API updates requires continuous developer learning.
Strong ecosystem, massive community, and widespread industry adoption.	JSX syntax learning curve for traditional web designers.
Easy debugging with React Developer Tools.	Documentation can lag behind frequent minor version releases.

1.4 Virtual DOM vs Real DOM Mechanism

Direct manipulation of the browser's **Real DOM** is computationally expensive because modifying a single node forces the browser to recalculate layout, recalculate CSS styles, and re-render the web page tree. React solves this bottleneck by introducing an in-memory **Virtual DOM (VDOM)**.

Aspect	Real DOM	Virtual DOM (React)
Update Speed	Slow and resource-intensive.	Fast and highly optimized in memory.
DOM Manipulation	Directly alters HTML element trees.	Updates in-memory tree first, then syncs.
Memory Usage	High memory overhead per DOM node.	Minimal overhead; lightweight JS object tree.
Re-rendering Process	Forces full subtree recalculation and repaint.	**Diffing algorithm** identifies modified elements only.

> **EXAM ESSENTIAL: VIRTUAL DOM RECONCILIATION PROCESS**
>
> **Virtual DOM Step-by-Step Reconciliation:**
> 1. Whenever underlying data or state changes, React renders the entire UI into a new Virtual DOM representation.
> 2. React compares the *new Virtual DOM* with the *previous Virtual DOM snapshot* (a process called **Diffing**).
> 3. React calculates the minimum required changes (the batch patch).
> 4. React updates **ONLY** those specific modified nodes in the browser's Real DOM (a process called **Reconciliation**).

2. Setting Up React Environment & ES6 Prerequisites

2.1 React Environment Setup Workflows

React applications can be built either directly in a browser sandbox for rapid prototyping or via local toolchains for production development.

Environment	Prerequisites	Installation / Execution Steps
Browser Playground	None (No installation required)	Use **https://codesandbox.io** to write and test React code directly in the browser.
Local Machine (Production)	Node.js & NPM installed	Execute the official npx create-react-app toolchain in terminal.

2.2 Local Terminal Command Sequence

Terminal Commands

```
# Step 1: Initialize new React application project folder
npx create-react-app my-app

# Step 2: Navigate into project directory
cd my-app

# Step 3: Start local development web server (runs on http://localhost:3000)
npm start
```

2.3 Essential ES6 JavaScript Refresher

Modern React development relies heavily on ECMAScript 2015 (ES6+) features. Mastering these syntax rules is compulsory for React exams:

ES6 Feature	Syntax Example	Usage in React
var vs let vs const	const name = 'React'; let count = 0;	const for immutable values/components; let for re-assignable variables.
Arrow Functions	const greet = () => Hi ;	Clean, concise component definitions and inline event handler callbacks.
Destructuring	const { name, age } = props;	Extracting properties directly from props or state objects.
Spread Operator (...)	<Child {...props} />	Forwarding all properties to inner components or copying objects/arrays.
Classes & Inheritance	class App extends React.Component	Defining traditional class-based React components.

2.4 Application Entry Point Templates

React 18 introduced a new client rendering API using createRoot, replacing the legacy ReactDOM.render method.

src/index.js (React 18 Standard)

```
// index.js — Modern React 18 Entry Point (RECOMMENDED)
import React from 'react';
import ReactDOM from 'react-dom/client';

// 1. Target the root HTML element in public/index.html
const root = ReactDOM.createRoot(document.getElementById('root'));

// 2. Render React element tree
root.render(&lt;h1&gt;Hello world from React&lt;/h1&gt;);
```

src/index.js (Legacy)

```
// index.js — Legacy Pre-React 18 Entry Point (For Historical Awareness)
import React from 'react';
import ReactDOM from 'react-dom';

ReactDOM.render(&lt;h1&gt;Hello World&lt;/h1&gt;, document.getElementById('root'));
```

3. Introducing JSX (JavaScript XML)

3.1 What is JSX?

JSX stands for **JavaScript XML**. It is a syntax extension for JavaScript that allows developers to write HTML-like tags directly inside JavaScript files. JSX is neither a string nor pure HTML. Under the hood, build tools (like Babel) compile JSX tags into standard JavaScript React.createElement() calls.

3.2 Embedding Dynamic Expressions

Any valid JavaScript expression (variables, function calls, arithmetic calculations) can be embedded inside JSX by enclosing it in **curly braces { }**.

src/index.js (JSX Expression Example)

```
import React from 'react';
import ReactDOM from 'react-dom/client';

const universityName = 'darshan university - rajkot';
const root = ReactDOM.createRoot(document.getElementById('root'));

// Embedding variable universityName inside JSX
root.render(&lt;h1&gt;Hello world from { universityName }&lt;/h1&gt;);
```

IMPORTANT JSX SYNTAX RULES

Core JSX Rules to Memorize for Exams:
1. **Single Parent Element:** JSX expressions must return a single root element (wrap multiple tags in or React Fragment <>...</>).
2. **Self-Closing Tags:** Empty XML/HTML elements MUST be explicitly closed (e.g., <img />, <input />,
).
3. **Reserved Keyword Renaming:** Use className instead of class, and htmlFor instead of for.
4. **camelCase Attributes:** JSX attributes use camelCase (e.g., onClick, tabIndex).

4. React Components & Component Lifecycle

4.1 Component Architecture & Capitalization Rule

Components let you split the user interface into independent, reusable pieces, and think about each piece in isolation. Conceptually, components act like JavaScript functions: they accept inputs (called **props**) and return React elements describing what should appear on screen.
STRICT RULE: Component names **MUST always start with a Capital Letter** (e.g., Welcome, not welcome). React treats lower-case elements as standard HTML DOM tags (e.g., , <h1>).

4.2 Function Components vs Class Components

1. Function Component Syntax (Standard & ES6 Arrow):

Function Component Syntax

```
// Standard Function Component
function Welcome(props) {
  return &lt;h1&gt;Hello world from { props.name }&lt;/h1&gt;;
}

// Equivalent ES6 Arrow Function Component
const Welcome = (props) =&gt; {
  return &lt;h1&gt;Hello world from { props.name }&lt;/h1&gt;;
};

root.render(&lt;Welcome name="darshan" /&gt;);
```

2. Class Component Syntax:

Class Component Syntax

```
// Class component extending React.Component
class Welcome extends React.Component {
  render() {
    // Accessing props via built-in this.props object
    return &lt;h1&gt;Hello world from { this.props.name }&lt;/h1&gt;;
  }
}

root.render(&lt;Welcome name="darshan" /&gt;);
```

4.3 Modular File Structure & Exports

In production React applications, each component is placed in its own separate file ending in .js or .jsx. The component is exported using export default ComponentName; and imported into other files without specifying the .js extension.

Modular Component Files (Welcome.js & index.js)

```
// File 1: src/Welcome.js
import React from 'react';

function Welcome() {
  return &lt;h1&gt;Hello World from other file&lt;/h1&gt;;
}
export default Welcome; // Default export

// ------------------------------------------------
// File 2: src/index.js
import React from 'react';
import ReactDOM from 'react-dom/client';
import Welcome from './Welcome'; // Omit .js extension

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(&lt;Welcome /&gt;);
```

4.4 Component Lifecycle Methods Breakdown

Every React component undergoes a lifecycle monitored and manipulated through built-in methods. The lifecycle is divided into **three main phases**:

Lifecycle Phase	Order of Execution	Method Name & Role / Description
1. Mounting (Initialization & Insertion into DOM)	1 2 3 4	constructor(): Initializes state & binds event handlers. getDerivedStateFromProps(): Syncs state with prop changes. render(): Returns JSX markup to be rendered. componentDidMount(): Executed immediately after component mounts; ideal for API calls.
2. Updating (Re-rendering triggered by Props or State changes)	1 2 3 4 5	getDerivedStateFromProps(): Called during update phase. shouldComponentUpdate(): Returns true/false to control re-rendering. render(): Re-evaluates component JSX UI. getSnapshotBeforeUpdate(): Captures DOM state before update. componentDidUpdate(): Executed immediately after DOM update is flushed.
3. Unmounting (Removal from DOM)	1	componentWillUnmount(): Executed right before component is destroyed; used for cleanup (canceling timers, listeners).

5. React Props (Properties)

5.1 What are Props?

Props (short for *properties*) are read-only inputs passed from a parent component to a child component. Props allow components to communicate with each other and enforce a strict **top-down (unidirectional) data flow**. While props resemble HTML attributes, they can pass any valid JavaScript value including **strings, numbers, objects, arrays, and callback functions**.

5.2 Props Access, Destructuring & Default Values

src/app.jsx (Props Destructuring & Default Props)

```
// app.jsx — Props Access, Parameter Destructuring &amp;amp; Default Value
import React from 'react';

// Component receiving destructured props with default parameter age=25
function InnerComponent({ name, age = 25 }) {
  return (
    &amp;lt;&amp;gt;
      &amp;lt;h1&amp;gt;Name = {name}&amp;lt;/h1&amp;gt;
      &amp;lt;h1&amp;gt;Age = {age}&amp;lt;/h1&amp;gt;
    &amp;lt;/&amp;gt;
  );
}

function App() {
  return (
    // Age prop is omitted here, so it falls back to default value 25
    &amp;lt;InnerComponent name="Dharmik" /&amp;gt;
  );
}
export default App;
```

5.3 Forwarding Props with JSX Spread Syntax ({...props})

When passing props down through multiple component levels, manually repeating prop names becomes repetitive and prone to error. The ES6 **spread operator ({...props})** allows forwarding all props to a nested component cleanly.

src/app.jsx (Forwarding Props with Spread Operator)

```
// app.jsx — Forwarding props using JSX Spread syntax ({...props})
function Inner2({ name, age }) {
  return (
    &amp;lt;&amp;gt;
      &amp;lt;h1&amp;gt;Name = {name}&amp;lt;/h1&amp;gt;
      &amp;lt;h1&amp;gt;Age = {age}&amp;lt;/h1&amp;gt;
    &amp;lt;/&amp;gt;
  );
}

// Inner1 receives props object and forwards all key-value pairs to Inner2
function Inner1(props) {
  return &amp;lt;Inner2 {...props} /&amp;gt;;
}

function App() {
  return &amp;lt;Inner1 name="Dharmik" age="25" /&amp;gt;;
}
export default App;
```

6. Event Handling in React

6.1 HTML DOM Events vs React JSX Events

Handling events in React is syntactically similar to handling events on HTML DOM elements, but with two key differences:

Feature / Syntax	Standard HTML DOM Event	React JSX Event
Event Naming Convention	Lower-case string (e.g., onclick, onchange)	**camelCase** naming (e.g., onClick, onChange, onDblClick)
Event Handler Assignment	Passes inline JavaScript string onclick="activateLasers()"	Passes actual **function reference** in curly braces onClick={activateLasers}

6.2 Passing Arguments to Event Handlers

To pass parameters/arguments into an event handler function, wrap the call inside an **inline arrow function callback**. Writing onClick={activateLasers(param)} directly would immediately execute the function upon component rendering rather than on click!

JSX Event Argument Passing

```
// Correct way to pass arguments to an event handler in JSX
&lt;button onClick={() =&gt; activateLasers(parameters)}&gt;
  Activate Lasers
&lt;/button&gt;
```

7. Conditional Rendering & Array Mapping

7.1 Conditional Rendering Techniques

In React, you can conditionally render specific component UI subtrees depending on application state or props. The two primary techniques covered in course materials are standard **if-else statements** and the **logical AND (&&) operator**.

Conditional Rendering Techniques

```
// Technique 1: Standard JavaScript if-else statement
function Welcome(props) {
  const isComputer = props.isComputer;
  if (isComputer) {
    return &lt;h1&gt;Computer Dept.&lt;/h1&gt;;
  } else {
    return &lt;h1&gt;Other Dept.&lt;/h1&gt;;
  }
}

// Technique 2: Inline Logical &amp;&amp; Operator (Short-Circuit Evaluation)
function WelcomeShort(props) {
  const isComputer = props.isComputer;
  return (
    &lt;&gt;
      {isComputer &amp;&amp; &lt;h1&gt;Comp Dept.&lt;/h1&gt;}
      {!isComputer &amp;&amp; &lt;h1&gt;Other Dept.&lt;/h1&gt;}
    &lt;/&gt;
  );
}
```

7.2 Array List Rendering using .map() Method

To dynamic render lists of data in React, JavaScript's built-in .map() array method is used. The .map() method iterates over each element in an array and transforms it into a new array of JSX React elements.

Array List Rendering using .map()

```
// JavaScript .map() Concept Refresher:
const numbers = [5, 3, 4];
const doubled = numbers.map((item) =&gt; item * 2); // doubled is [10, 6, 8]

// React Component rendering list of JSX elements using .map():
function Welcome() {
  const names = ["Dharmik Aghera", "darshan", "university"];

  // Map each name string into an &lt;h1&gt; React element
  const nameElements = names.map((n) =&gt; &lt;h1&gt;Welcome from {n}&lt;/h1&gt;);

  return nameElements;
}
```

8. Exam Preparation Kit

8.1 High-Yield Exam Points & Key Rules

- **React Definition:** Declarative, efficient, and flexible front-end JS library created by Facebook for SPAs.
- **Virtual DOM Reconciliation:** Virtual DOM detects changes via *diffing algorithm* and updates *only* changed nodes in Real DOM.

- **Component Naming:** Component names **MUST** start with an uppercase letter.
- **Function vs Class Components:** Function components + Hooks are modern standard; Class components extend React.Component and require render() method.
- **Props Mutability:** Props are strictly **read-only** immutable inputs passed from parent to child.
- **Event Syntax:** React events use camelCase (e.g. onClick) and receive function references.
- **Lifecycle Phases:** Mounting, Updating, Unmounting (10 key lifecycle methods total).

8.2 Likely Subjective Exam Questions & Model Answers

Q1: Explain Virtual DOM and how it improves React performance over Real DOM. (3-5 Marks)

Model Answer: Real DOM manipulation is slow because modifying HTML elements triggers costly layout recalculations and repaints. React creates a lightweight in-memory Virtual DOM tree. When underlying data or state changes, React renders a new Virtual DOM, compares it with the previous snapshot using a fast *diffing algorithm*, and updates **only** the modified nodes in the Real DOM. This batching and selective updating minimizes browser reflows and significantly boosts application performance.

Q2: Differentiate between Function Components and Class Components in React. (3 Marks)

Model Answer:
1. *Syntax:* Function components are plain JS functions returning JSX. Class components extend React.Component and require a render() method.
2. *Props Access:* Function components receive props as function arguments. Class components access props via this.props.
3. *Industry Trend:* Function components with React Hooks (introduced in React 16.8) are the current standard recommended by React.

Q3: Write a React component to render a list of 3 university names using array map. (4 Marks)

Model Code Answer — List Rendering

```
import React from 'react';

function UniversityList() {
  const universities = ['Darshan University', 'GTU', 'NIRMA'];
  return (
    &lt;div&gt;
      &lt;h2&gt;University Directory&lt;/h2&gt;
      &lt;ul&gt;
        {universities.map((uni, index) =&gt; (
          &lt;li key={index}&gt;{uni}&lt;/li&gt;
        ))}
      &lt;/ul&gt;
    &lt;/div&gt;
  );
}
export default UniversityList;
```

8.3 Multiple Choice Questions (MCQs) with Explanations

PRACTICE MCQ

1. What is ReactJS?
A) A server-side database
B) A front-end JavaScript library for building user interfaces
C) A CSS layout framework
D) A back-end Python framework

Correct Answer: B
Explanation: ReactJS is an open-source front-end JavaScript library created by Facebook for constructing user interfaces and SPAs.

PRACTICE MCQ

2. Which command is used to set up a new React app locally?
A) npm install react-app
B) npx create-react-app my-app
C) node start react
D) react-cli new app

Correct Answer: B
Explanation: npx create-react-app my-app is the standard command provided by React to scaffold a complete project.

PRACTICE MCQ

3. Component names in React MUST start with:
A) A lowercase letter
B) An uppercase letter
C) A dollar sign ($)
D) An underscore (_)

Correct Answer: B
Explanation: React requires component names to start with an uppercase letter to distinguish them from standard HTML tags.

PRACTICE MCQ

4. How do you pass a variable value inside JSX tags?
A) Using double quotes "var"
B) Using curly braces {var}
C) Using parenthetical (var)
D) Using angle brackets

Correct Answer: B
Explanation: Curly braces { } are used in JSX to embed any valid JavaScript expression or variable.

PRACTICE MCQ

5. Which method is executed immediately after a component is mounted into the DOM?
A) componentWillMount()
B) componentDidMount()
C) render()
D) shouldComponentUpdate()

Correct Answer: B
Explanation: componentDidMount() is invoked right after a component is inserted into the DOM tree.

9. Final Revision Sheet

9.1 Core Terms Cheat Sheet

Term	One-Sentence Definition / Exam Formula
ReactJS	Declarative front-end JS library for UI & SPA building.
Virtual DOM	In-memory lightweight DOM copy used to diff and update Real DOM efficiently.
JSX	JavaScript XML syntax extension enabling HTML tags inside JS code.
Component	Independent, reusable modular UI building block.
Props	Read-only immutable inputs passed from parent to child component.
Diffing	Algorithm process comparing new Virtual DOM snapshot with old snapshot.
Reconciliation	Process of syncing Virtual DOM batch changes into Real DOM.
.map()	JS array method used to transform data arrays into lists of JSX elements.

9.2 Master Structured Comparison Tables

Comparison 1: Virtual DOM vs Real DOM

Feature	Virtual DOM	Real DOM
Performance	Extremely fast execution.	Slow DOM manipulations.
Update Logic	Updates only changed nodes (Diffing).	Re-renders entire element subtrees.
Memory Impact	Minimal in-memory JS object overhead.	High memory usage per browser element.
Direct Editing	Cannot directly edit HTML page.	Directly edits HTML elements.

Comparison 2: Function Components vs Class Components

Feature	Function Component	Class Component
Declaration	Plain JS function or arrow function.	ES6 class extending React.Component.
Props Access	Passed as parameter (props or {propName}).	Accessed via this.props.

Feature	Function Component	Class Component
Render Method	Directly returns JSX elements.	Requires render() method returning JSX.
Modern Industry Use	Recommended standard (with Hooks).	Legacy codebase support.

Comparison 3: React Props vs Component State

Aspect	Props (Properties)	State
Mutability	**Immutable** (Read-only inside child).	**Mutable** (Can be updated inside component).
Data Source	Passed down from Parent component.	Managed internally within component.
Purpose	Component communication & configuration.	Managing dynamic UI changes over time.
Syntax	props.name / {name}	Managed via useState Hook / this.state

FINAL EXAM WISDOM

BEST OF LUCK FOR YOUR EXAM!
Remember to write neat code syntax, capitalize component names, use camelCase for event handlers, and mention Virtual DOM reconciliation whenever performance questions appear on the test paper.